How-To Library

Informatica

Performance Guidelines for Databricks for PowerExchange for Microsoft Azure Blob Storage

© Copyright Informatica LLC 2019, 2021. Informatica, the Informatica logo, and PowerExchange are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

When you use PowerExchange for Microsoft Azure Blob Storage to read data from or write data to Microsoft Azure Blob Storage, multiple factors such as hardware parameters, database parameters, application server parameters, and Informatica mapping parameters impact the adapter performance. You can optimize the performance by tuning these parameters appropriately. This article describes general reference guidelines to help you tune the performance of PowerExchange for Microsoft Azure Blob Storage for Azure Databricks.

Supported Versions

- PowerExchange for Microsoft Azure Blob Storage 10.2.2

Table of Contents

Overview 2
Best Practices for Tuning Performance. 2
Tune the Mapping 3
Ports Precision. 3
Data Type Mapping. 3

Overview

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance and monitor and adjust the results as required.

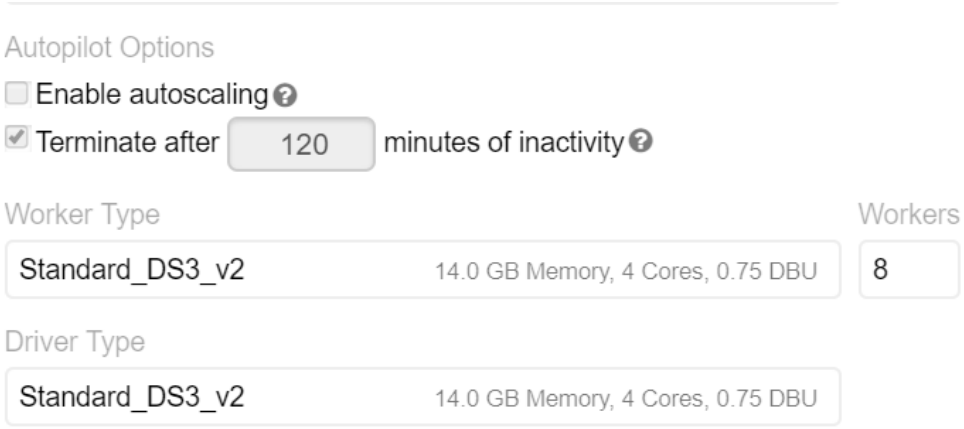
This document describes the key hardware, Databricks cluster and Informatica mapping parameters that you can tune to optimize the performance of PowerExchange for Microsoft Azure Blob Storage on Microsoft Azure Databricks cluster.
Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of PowerExchange for Microsoft Azure Blob Storage might vary based on individual environments and other parameters even when you use the same data.

Best Practices for Tuning Performance

Consider the following best practices for better performance:

- Reading or writing AVRO files is faster than reading or writing flat files on Databricks environment.
- The Domain Host, Source, Target and Databricks cluster should be in the same region to avoid network latency.
- General-purpose V2 provides better computing and networking and is a recommended storage type by Microsoft.
- Ensure that **Enable autoscaling** is not selected while configuring the Databricks cluster.
- Maximum number of worker nodes are available to complete the job while configuring the Databricks cluster.

The following image describes how to disable autoscaling and provide number of worker nodes before launching the Databricks cluster:



- Based on the resource requirement, choose the appropriate instance type for Worker Type and Driver Type.

  For more information on pricing and resource details for various instance types offered, see https://azure.microsoft.com/en-in/pricing/details/databricks/.

Tune the Mapping

You can optimize the performance of PowerExchange for Microsoft Azure Blob Storage on Microsoft Azure Databricks by tuning the mapping.

You can tune the following parameters at the mapping level in the Developer Tool:

- Ports precision
- Data type mapping

For more information, see the *Informatica Performance Tuning Guide*.

Ports Precision

Precision is the maximum number of significant digits for numeric data types, or maximum number of characters for string data types. For numeric data types, precision includes scale.

You can tune the precision in Model repository mappings.

When mappings contain ports with a larger precision than required, the mapping performance degrades. Informatica recommends that you set the precision judiciously for all source ports, transformation ports, and target ports. For instance, if a string port can handle data of a maximum of 200 characters, set the precision to 200. Do not set the precision to a high value such as 1000.

Data Type Mapping

When the Integration Service reads source data, it converts the native data types to the comparable transformation data types before transforming the data. When the Integration Service writes data to a target, it converts the transformation data types to the comparable native data types.

When you map source ports to transformation ports and then to target ports, avoid unnecessary data type conversions. For instance, do not map a port of the string data type to a port of the date data type. Ensure that you map ports to the same data type in all components of the mapping. Also, remove all unconnected ports from the mapping.

Authors

Krishna Prabhakar Devarakonda

Manav Khurana

Anushka Bajaj